

PODSTAWY GLSL

Praktyczna ściągą: pipeline, typy, kwalifiery, funkcje wbudowane, podpięcie do Three.js. Zwiężle, do druku.

01 CZYM JEST SHADER

Shader to mały program w języku **GLSL** (OpenGL Shading Language) wykonywany na GPU. Piszę się go w stylu C: silne typowanie, brak garbage collector, brak rekurencji, brak wskaźników. Shader działa dla *każdego wierzchołka* i *każdego piksela* równolegle, dlatego musi być prosty i lokalny.

W praktyce zawsze piszesz dwa shadery, które razem tworzą program:

- **Vertex shader** — uruchamia się raz na każdy wierzchołek geometrii. Jego praca: wyznaczyć `gl_Position`, czyli pozycję wierzchołka w przestrzeni clip-space.
- **Fragment shader** — uruchamia się raz na każdy widoczny piksel (poprawnie: *fragment*). Jego praca: wyznaczyć kolor — `gl_FragColor` w GLSL ES 1.00, lub własny `out vec4` w GLSL ES 3.00.

REGUŁA KCIUKA

Vertex shader rusza geometrią. Fragment shader maluje piksele. Wszystko, co dotyczy kształtu — vertex. Wszystko, co dotyczy koloru — fragment.

02 PIPELINE I PRZEPŁYW DANYCH

Shader nie działa w próżni. Three.js (przez WebGL) wstrzykuje do niego dane na trzy sposoby, oznaczone **qualifierami**:

QUALIFIER	SKĄD	ZMIENNOŚĆ	PRZYKŁAD
<code>attribute</code>	JS → vertex	per-wierzchołek	<code>position</code> , <code>uv</code> , <code>normal</code>
<code>uniform</code>	JS → vertex i fragment	stałe w obrębie draw call	<code>u_time</code> , <code>projectionMatrix</code>
<code>varying</code>	vertex → fragment	interpolowane między wierzchołkami	<code>vUv</code> , <code>vNormal</code>

Three.js automatycznie deklaruje wiele atrybutów i uniformów dla `ShaderMaterial` (m.in. `position`, `uv`, `normal`, `modelViewMatrix`, `projectionMatrix`, `cameraPosition`). Nie trzeba ich definiować w GLSL — wystarczy ich używać. `RawShaderMaterial` tego nie robi.

03 SKĄD PRZYCHODZĄ DANE — KROK PO KROKU

Tabelka wyżej pokazuje kwalifiery, ale nie pokazuje *kierunków*. To jest najczęstsze nieporozumienie początkujących, więc rozłożmy to wprost. Są **dwie niezależne ścieżki** danych do shaderów, nie jedna.



Czyli:

- **Uniform** pochodzi z JS bezpośrednio do obu shaderów, niezależnie. Vertex shader nie pośredniczy w przekazywaniu uniformów do fragmenta — JS rozsyła je równolegle.
- **Varying** jest **jedyną ścieżką między vertex a fragment shaderem**. Vertex pisze, fragment czyta. Po drodze GPU automatycznie interpoluje wartości między wierzchołkami.
- **Attribute trafia tylko do vertex shadera** — fragment shader nigdy nie widzi atrybutów bezpośrednio. Jeśli chcesz mieć UV w fragmencie, vertex musi je przekazać przez varying.

CZĘSTY BŁĄD

„Uniformy pochodzą z vertex shadera”. Nie. Uniform = stała z JS. Varying = przesyłka z vertexa do fragmenta. Mylenie tego prowadzi do dziwnych prób „dodawania” uniforma w vertex shaderze, żeby był „dostępny” w fragmencie.

TEN SAM UNIFORM W OBU SHADERACH

Vertex i fragment shader to dla GPU dwa **niezależne** programy. Jeśli chcesz używać tego samego uniforma w obu, musisz go zadeklarować w każdym osobno. JS ustawia go raz, ale każdy shader potrzebuje własnej deklaracji — to są jakby dwa „abonamenty” na tę samą wartość.

```
// vertex shader
uniform float u_time;
varying vec2 vUv;
void main() {
    vUv = uv;
    vec3 pos = position;
    pos.z += sin(u_time); // używam u_time tutaj
    gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}

// fragment shader
uniform float u_time; // ten sam uniform, deklarowany niezależnie
varying vec2 vUv;
void main() {
    gl_FragColor = vec4(vec3(sin(u_time)), 1.0); // i tutaj też
}
```

Po stronie JS — jedna linia, obaj widzą:

```
material.uniforms.u_time.value = clock.getElapsedTime();
```

HEURYSTYKA DO ZAPAMIĘTANIA

`uniform` = stała z JS, identyczna dla wszystkich pikseli/wierzchołków w jednym wywołaniu rysowania. `varying` = przesyłka z vertexa do fragmenta, interpolowana po drodze. `attribute` = dane per-wierzchołek z geometrii.

04 TYPY W GLSL

GLSL jest silnie typowany. Konwersja niejawną jest minimalna — `1` to `int`, a `1.0` to `float`. Mieszanie ich kończy się błędem kompilacji.

TYP	CO TO	PRZYKŁAD UŻYCIA
<code>float</code>	liczba zmiennoprzecinkowa	<code>float t = 1.5;</code>
<code>int</code> , <code>bool</code>	całkowita, logiczna	<code>int n = 4;</code>
<code>vec2</code> , <code>vec3</code> , <code>vec4</code>	wektor 2/3/4 floatów	<code>vec3 c = vec3(1.0, 0.5, 0.0);</code>
<code>mat2</code> , <code>mat3</code> , <code>mat4</code>	macierz	<code>mat4 m = projectionMatrix;</code>
<code>sampler2D</code>	uchwyt do tekstury	<code>texture2D(tex, uv)</code>

SWIZZLING — NAJSŁODSZA CECHA GLSL

Komponenty wektora można czytać i przestawiać po nazwach: `.x .y .z .w` (pozycja), `.r .g .b` (kolor), `.s .t .p .q` (tekstura). Wszystkie są równoważne, używaj tej, która niesie znaczenie.

```
vec4 c = vec4(0.2, 0.5, 0.8, 1.0);
vec3 rgb  = c.rgb;           // 0.2, 0.5, 0.8
vec3 bgr  = c.bgr;          // odwrotnie
vec2 xy   = c.xy;
vec4 rrrr = c.rrrr;         // powielenie kanału
float alpha = c.a;
```

05 MINIMALNY SHADER

Najprostsza para, która coś robi: vertex przepuszcza pozycję przez macierze Three.js, fragment maluje na pomarańczowo.

VERTEX SHADER

```
varying vec2 vUv;

void main() {
    vUv = uv;
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
}
```

FRAGMENT SHADER

```
precision mediump float;
varying vec2 vUv;
uniform float u_time;

void main() {
    vec3 col = vec3(vUv.x, vUv.y, 0.5 + 0.5 * sin(u_time));
    gl_FragColor = vec4(col, 1.0);
}
```

06 PODPIĘCIE W THREE.JS

```
import * as THREE from 'three';

const material = new THREE.ShaderMaterial({
  uniforms: {
    u_time: { value: 0.0 }
  },
  vertexShader,    // stringi z .glsl
  fragmentShader,
});

const mesh = new THREE.Mesh(
  new THREE.PlaneGeometry(2, 2, 128, 128),
  material
);
scene.add(mesh);

// w pętli renderującej
material.uniforms.u_time.value = clock.getElapsedTime();
```

PUŁAPKA POZĄTKUJĄCEGO

Jeśli zwiększasz tylko rozmiar PlaneGeometry, ale zostawiasz 1, 1 w segmentach, twój vertex shader nie ma czym ruszać — masz 4 wierzchołki na całą płaszczyznę. Daj 128, 128 jak chcesz robić displacement.

07 FUNKCJE, KTÓRE MUSISZ ZNAĆ

MATEMATYKA

- `sin(x)`, `cos(x)`, `tan(x)`
- `pow(x, y)`, `sqrt(x)`, `exp(x)`
- `abs(x)`, `sign(x)`, `floor(x)`, `fract(x)`
- `mod(x, y)` — modulo, ale dla floatów
- `min(a, b)`, `max(a, b)`, `clamp(x, lo, hi)`

INTERPOLACJA

- `mix(a, b, t)` — liniowy lerp
- `step(edge, x)` — twarde 0/1
- `smoothstep(e0, e1, x)` — gładkie 0–1
- `length(v)`, `distance(a, b)`
- `normalize(v)`, `dot(a, b)`, `cross(a, b)`

TRZY NAJWAŻNIEJSZE TRICKI

```
// 1. Pulsowanie 0..1 z sin (sam sin daje -1..1)
float p = 0.5 + 0.5 * sin(u_time);

// 2. Miękka krawędź zamiast if-a
float mask = smoothstep(0.4, 0.5, length(vUv - 0.5));

// 3. Powtarzalny wzór z fract
vec2 tiled = fract(vUv * 10.0); // 10x10 kafelków
```

08 PRECYZJA

W GLSL ES (WebGL) trzeba zadeklarować precyzję dla floatów we fragment shaderze. Three.js robi to za ciebie w `ShaderMaterial`, ale warto rozumieć:

- `lowp` — szybka, mało dokładna. Do kolorów wystarczy.
- `mediump` — domyślny rozsądny wybór.
- `highp` — wymagana dla obliczeń pozycji w dużej skali (np. raymarching).

09 NAJCZĘSTSZE BŁĘDY

SYMPTOM	PRZYCZYNA
Shader się nie kompiluje, ale błąd niejasny	Mieszasz <code>int</code> z <code>float</code> . Pisz <code>1.0</code> nie <code>1</code> .
Czarny ekran	Vertex zwraca pozycję poza clip-space, fragment alpha=0, lub uniform nie jest aktualizowany w pętli.
Płaski kolor mimo displacementu	Geometria ma za mało wierzchołków — zwiększ segmenty.
Plama zamiast tekstury	Brakujący <code>varying vUv</code> lub nie przekazałeś go w vertex shaderze.
Działa lokalnie, nie na produkcji	Sterownik wymaga <code>precision highp float;</code> we fragmencie.

10 GDZIE IŚĆ DALEJ

Jak ogarniesz powyższe, naturalna ścieżka to: **noise** (Ashima webgl-noise), **FBM** (fractal brownian motion), **SDF i raymarching** (Inigo Quilez), a potem **TSL** — Three.js Shading Language jako bardziej współczesna alternatywa, kompilująca się i do GLSL, i do WGSL.



MATERIAŁY

Dwa źródła, którymi warto pójść po przeczytaniu tej ściagi:

KSIĄŻKA ONLINE · DARMOWE

THE BOOK OF SHADERS

Patricio Gonzalez Vivo i Jen Lowe. Najlepszy materiał wstępny do GLSL w sieci. Interaktywne przykłady, każdy rozdział dorzuca jedną technikę. Zaczynij od „Hello World”, przez funkcje kształtujące, do generative patterns i noise.

<https://thebookofshaders.com/>

PLAYGROUND · LIVE CODING

CODEPEN — SZABLON THREE.JS + GLSL

Świeży pen z gotowym ShaderMaterial, uniformami i animacją. Forkuj i baw się. Dobry do testów bez stawiania projektu Vite.

<https://codepen.io/enginarslan/pen/PoZNQpd>

BONUS · GALERIA + EDYTOR

SHADERTOY

Tu pisze społeczność. Czytaj cudze shadery, klonuj, modyfikuj. Studium najlepszych Inigo Quileza i kishimisu da ci więcej niż dziesięć tutoriali.

<https://www.shadertoy.com/>